

Uniwersytet Wrocławski
Wydział Fizyki i Astronomii

Oliwer Urbaniak

System zarządzania pracą drukarki 3D w laboratorium
studenckim.

Praca inżynierska na kierunku
Informatyka Stosowana i Systemy Pomiarowe

Opiekun
dr Bartosz Brzostowski

Wrocław, 21 maja 2026

Spis treści

Streszczenie	7
1 Wprowadzenie	8
1.1 Motywacja i cel pracy	8
1.2 Zakres pracy	8
1.3 Struktura pracy	9
2 Analiza wymagań i przegląd rozwiązań	11
2.1 Wymagania funkcjonalne	11
2.2 Wymagania нефункционалне	12
2.3 Przegląd istniejących rozwiązań	13
2.3.1 OctoPrint	13
2.3.2 Mainsail oraz Fluidđ	13
2.3.3 Platformy komercyjne	14
2.3.4 Wnioski końcowe	14
3 Wybór technologii	15
3.1 Język programowania oraz środowisko uruchomieniowe	15
3.1.1 TypeScript oraz Node.js	15
3.1.2 Python	15
3.2 Framework webowy — Express.js	15
3.3 Baza danych — Azure Cosmos DB	16
3.4 Repozytorium obiektowe — Azure Blob Storage	16
3.5 Broker komunikatów — Azure Service Bus	16
3.6 Sterowanie urządzeniem — Azure IoT Hub	17
3.7 Interfejs użytkownika — React oraz Vite	17
3.8 Konteneryzacja — Docker oraz Docker Compose	18
3.9 CAD - SolidWorks	18
3.10 Porównanie wybranych alternatyw	19
4 Architektura systemu	22
4.1 Ogólna struktura	22

4.2	Diagram architektury	22
4.3	Model danych	24
4.3.1	Użytkownik (kontener users)	25
4.3.2	Zadanie do wydruku (kontener jobs)	26
4.3.3	Token odświeżania (kontener refresh-tokens)	26
4.4	Cykl życia zadania do wydruku	27
4.5	Bezpieczeństwo i uwierzytelnianie	28
5	Implementacja serwisów backendowych	30
5.1	Auth Service	30
5.1.1	Opis ogólny	30
5.1.2	Punkty końcowe API	30
5.1.3	Kluczowe fragmenty implementacji	30
5.1.4	Weryfikacja adresu poczty elektronicznej	33
5.2	User Service	33
5.2.1	Opis ogólny	33
5.2.2	Punkty końcowe API	34
5.2.3	Mechanizm autoryzacji żądań	35
5.3	Files Service	37
5.3.1	Opis ogólny	37
5.3.2	Procedura przetwarzania i przesyłania plików	37
5.4	Queue Service	39
5.4.1	Opis ogólny	39
5.4.2	Mechanizm nasłuchiwanie kolejki	39
5.4.3	Punkty końcowe HTTP API	41
5.5	Printer Service	41
5.5.1	Opis ogólny	41
5.5.2	Mechanizm harmonogramowania	42
5.5.3	Monitor gotowości urządzenia	44
5.5.4	Odbiornik telemetry	44
5.5.5	Punkty końcowe HTTP API	45
5.6	Video Service	47
5.6.1	Opis ogólny	47

5.6.2	Punkty końcowe API	47
6	Implementacja frontendu	48
6.1	Struktura aplikacji	48
6.2	Klient API	48
6.3	Routing i ochrona tras	49
6.4	Ekrany aplikacji	49
6.4.1	Strona główna	49
6.4.2	Dashboard użytkownika	49
6.4.3	Upload i planowanie	49
6.4.4	Kontrola druku	49
6.4.5	Panel administracyjny	49
6.5	Internacjonalizacja	49
6.6	Obsługa motywu	49
7	Agent Raspberry Pi - AddiPi Agent	50
7.1	Opis ogólny	50
7.2	Architektura agenta	50
7.3	Połączenie z IoT Hub	50
7.4	Obsługiwane metody bezpośrednie	50
7.5	Przebieg obsługi metody startPrint	50
7.6	Telemetria	50
7.7	Komunikacja z OctoPrint	50
8	Obudowa urządzenia - projekt CAD	51
8.1	Cel i zakres projektu mechanicznego	51
8.2	Moduł CASE - obudowa Raspberry Pi z kamerą	51
8.2.1	Podstawa projektowa	51
8.2.2	Zakres modyfikacji	51
8.2.3	Mocowanie kamery	51
8.2.4	Struktura plików CASE	51
8.3	Moduł STAND - podstawka regulowana	51
8.4	Złożenie całości	51
8.5	Parametry wydruku	51

8.6	Instrukcja montażu	51
9	Konfiguracja Raspberry Pi	52
9.1	Opis środowiska	52
9.2	Instalacja agenta	52
9.3	Automatyczne uruchamianie agenta przez systemd	52
9.4	Tunel Cloudflare - ekspozycja kamery	52
9.4.1	Instalacja cloudflared	52
9.4.2	Skrypt publikowania URL kamery	52
9.4.3	Cykliczne odnawianie URL	52
9.5	Konfiguracja crontab	52
9.5.1	Uzasadnienie parametrów	52
9.6	Problem z dostępnością sieci przy starcie	52
9.7	Podsumowanie konfiguracji Raspberry Pi	52
10	Wdrożenie i infrastruktura	53
10.1	Infrastruktura chmurowa	53
10.2	Inicjalizacja infrastruktury	53
10.3	Konteneryzacja - Dockerfiles	53
10.4	Docker Compose	53
10.5	Zmienne środowiskowe	53
10.6	Wdrożenie frontendu	53
10.7	Wdrożenie agenta	53
11	Testowanie systemu	54
11.1	Metodologia testowania	54
11.2	Testy jednostkowe - Auth Service	54
11.3	Testy integracyjne przez Postman	54
11.4	Test end-to-end - pełny przepływ druku	54
11.5	Testy wydajnościowe i obserwacje	54
12	Podsumowanie	55
12.1	Osiągnięcia	55
12.2	Napotkane trudności	55

12.3	Możliwości rozwoju	55
12.4	Wnioski	55
Załączniki		58
A Konfiguracja środowiska deweloperskiego		58
A.1	Wymagania	58
A.2	Pierwsze uruchomienie	58
B Struktura repozytoriów		58
C Endpointy API - podsumowanie		58

Streszczenie

Niniejsza praca inżynierska opisuje projekt, implementację oraz walidację rozproszonego systemu zarządzania procesami druku trójwymiarowego o nazwie *AddiPi*, dedykowanego dla akademickich laboratoriów studenckich. Opracowane rozwiązanie umożliwia użytkownikom zdalne przysyłanie plików z kodem sterującym (G-code), harmonogramowanie oraz monitorowanie stanu realizacji zadań w czasie rzeczywistym. Administratorom systemu udostępniono natomiast moduły pełnej kontroli nad kolejką produkcyjną, urządzeniami oraz profilami użytkowników.

Architektura systemu została oparta na paradygmacie mikroserwisowym. W strukturze backendowej wyodrębniono sześć niezależnych komponentów: usługę uwierzytelniania (*Auth Service*), zarządzania profilami (*User Service*), obsługi plików (*Files Service*), orkiestracji kolejki zadań (*Queue Service*), sterowania urządzeniem (*Printer Service*) oraz agenta sprzętowego (*AddiPi Agent*) uruchamianego na platformie Raspberry Pi. Warstwę prezentacji stanowi aplikacja kliencka (frontendowa) zbudowana z wykorzystaniem biblioteki React, którą uzupełnia dedykowany serwis wideo (*Video Service*) odpowiedzialny za dystrybucję strumienia wizyjnego.

Komunikacja wewnątrz systemu realizowana jest hybrydowo — za pomocą synchronicznego protokołu HTTP (architektura REST API) oraz asynchronicznej, chmurowej kolejki *Azure Service Bus*. Dane aplikacyjne są utrwalane w wielomodelowej bazie danych *Azure Cosmos DB*, a pliki G-code w repozytorium obiektowym *Azure Blob Storage*. Zarządzanie i sterowanie fizyczną drukarką 3D odbywa się za pośrednictwem usługi *Azure IoT Hub* przy użyciu mechanizmu metod bezpośrednich (ang. *Direct Methods*) wywoływanych na mikrokomputerze, na którym osadzono oprogramowanie serwerowe *OctoPrint*.

Poszczególne serwisy zaimplementowano w językach TypeScript (środowisko Node.js) oraz Python. Proces wdrożenia środowiska oparto na technologii konteneryzacji Docker, koordynowanej za pomocą narzędzia Docker Compose.

Słowa kluczowe: mikroserwisy, drukowanie 3D, OctoPrint, Raspberry Pi, Azure IoT Hub, Azure Cosmos DB, Docker, TypeScript, Python, REST API, systemy rozproszone.

1 Wprowadzenie

1.1 Motywacja i cel pracy

Drukarki 3D stanowią obecnie standardowe wyposażenie laboratoriów akademickich, warsztatów studenckich oraz zakładów produkcyjnych zorientowanych na wytwarzanie przyrostowe. Urządzenia te, pomimo rosnącej dostępności i malejących cen, pozostają zasobem współdzielonym — ich liczba w danej jednostce jest zwykle ograniczona, a czas wytwarzania pojedynczego modelu fizycznego może wynosić od kilkunastu minut do wielu godzin. W środowisku akademickim oraz przemysłowym rodzi to naturalne wyzwania logistyczne i organizacyjne, takie jak konieczność ręcznego rezerwowania terminów, brak przejrzystości w strukturze kolejki zadań oraz brak możliwości zdalnego monitorowania stanu operacyjnego maszyn.

Głównym celem niniejszej pracy jest zaprojektowanie, implementacja oraz walidacja rozproszonego systemu *AddiPi*, którego zadaniem jest pełna automatyzacja i orkiestracja procesów zarządzania drukarkami 3D w środowisku laboratorium studenckiego. Opracowane oprogramowanie udostępnia użytkownikom funkcjonalności w zakresie:

- asynchronicznego przesyłania plików z kodem sterującym (G-code) za pośrednictwem aplikacji klienckiej,
- elastycznego harmonogramowania procesów druku na określony termin lub lokowania ich w ogólnej kolejce systemu,
- śledzenia bieżących metryk postępu prac w czasie rzeczywistym wraz z bezpośrednim podglądem wizyjnym,
- automatycznego otrzymywania powiadomień o pomyślnym zakończeniu bądź awarii procesu produkcyjnego.

Personelowi administracyjnemu laboratorium system zapewnia dedykowane narzędzia do zarządzania profilami użytkowników i ich uprawnieniami, audytu historii wszystkich operacji oraz zdalnego sterowania infrastrukturą sprzętową i priorytetami kolejki.

1.2 Zakres pracy

Zakres rzeczowy zrealizowanego projektu inżynierskiego obejmuje:

1. przeprowadzenie analizy wymagań funkcjonalnych i нефункциональных oraz wybór technologii,
2. opracowanie projektu rozproszonej architektury mikroserwisowej,
3. implementację komponentów serwerowych (backendowych) oraz aplikacji klienckiej (frontendowej),
4. konfigurację i integrację dedykowanej infrastruktury chmurowej w modelu PaaS (*Microsoft Azure*),
5. konteneryzację oraz automatyzację procesu wdrożenia systemu przy użyciu narzędzi *Docker* oraz *Docker Compose*,
6. przeprowadzenie testów weryfikacyjnych i walidacji poprawności działania całego ekosystemu.

1.3 Struktura pracy

Niniejsza praca dyplomowa została podzielona na logiczne segmenty, których zawartość odzwierciedla kolejne etapy realizacji projektu. Rozdział 2 zawiera szczegółową analizę wymagań funkcjonalnych i нефункциональных oraz krytyczny przegląd istniejących rozwiązań rynkowych. W rozdziale 3 omówiono wybrane technologie programistyczne i chmurowe wraz z technicznym uzasadnieniem podjętych decyzji projektowych. Projekt samej architektury rozproszonej oraz przepływy danych w systemie zostały opisane w rozdziale 4.

Szczegółowa implementacja poszczególnych komponentów programowych, z uwzględnieniem warstwy serwerowej, klienckiej oraz komponentów pomocniczych, stanowi treść rozdziałów od 5 do 7. Z kolei w rozdziale 8 przedstawiono proces projektowania mechanicznego i trójwymiarowego modelowania obudowy urządzenia w programie *SolidWorks*. Specyfika konfiguracji mikrokomputera sterującego, automatyzacja zadań systemowych przez mechanizm *cron* oraz konfiguracja bezpiecznego tunelu sieciowego *Cloudflare* zostały przybliżone w rozdziale 9.

Końcowe etapy prac, obejmujące orkiestrację infrastruktury chmurowej *Microsoft Azure* oraz procesy konteneryzacji środowiska, opisano w rozdziale 10. Rezultaty przeprowadzonych testów weryfikacyjnych oraz walidację poprawności działania gotowego eko-

systemu zaprezentowano w rozdziale 11. Całość pracy kończy się podsumowaniem oraz wnioskami końcowymi, zawartymi w rozdziale 12.

2 Analiza wymagań i przegląd rozwiązań

2.1 Wymagania funkcjonalne

Na podstawie szczegółowej analizy specyfiki oraz potrzeb akademickiego laboratorium studenckiego sformułowano następujące wymagania funkcjonalne, z podziałem na role użytkownika standardowego oraz administratora systemu.

W obszarze kont użytkowników (rola standardowa):

- rejestracja konta powiązana z dwuetapową weryfikacją adresu e-mail, ograniczoną wyłącznie do domeny akademickiej @uwr.edu.pl,
- bezpieczne uwierzytelnianie i autoryzacja z wykorzystaniem mechanizmu podwójnych tokenów JWT (*access token* oraz *refresh token*),
- asynchroniczne przesyłanie plików z kodem sterującym (G-code) o maksymalnym rozmiarze pojedynczego wolumenu do 50 MB i rozszerzeniu .gcode,
- elastyczne harmonogramowanie procesów wytwórczych na wybrany termin kalendarzowy lub lokowanie ich w ogólnej kolejce systemu,
- przeglądanie historii własnych zleceń produkcyjnych z opcją dynamicznego filtrowania według stanów maszyny,
- monitorowanie parametrów środowiskowych i postępu procesu druku 3D w czasie rzeczywistym (weryfikacja paska postępu, temperatury dyszy oraz stołu roboczego, a także podgląd wizyjny z kamery),
- możliwość manualnego anulowania lub ponownego inicjowania własnych zadań.

W obszarze administracji i nadzoru (rola administratora):

- wgląd w rejestr kont oraz zarządzanie profilami wszystkich użytkowników systemu,
- dynamiczna modyfikacja poziomu uprawnień i ról użytkowników (nadawanie statusu user lub admin),
- kompleksowe zarządzanie globalną kolejką oraz historią wszystkich zarejestrowanych zadań drukowania,

- zdalne potwierdzanie fizycznej i operacyjnej gotowości urządzenia do podjęcia kolejnego cyklu produkcyjnego,
- dostęp do zagregowanych metryk systemowych, obejmujących m.in. statystyki zadań z podziałem na statusy.

2.2 Wymagania niefunkcjonalne

W celu zapewnienia odpowiedniej jakości, wydajności oraz stabilności działania rozproszonej platformy informatycznej, określono następujące wymagania jakościowe (niefunkcjonalne):

- **Skalowalność** — architektura rozproszona powinna umożliwiać bezproblemową integrację i rejestrację kolejnych fizycznych urządzeń drukujących bez konieczności modyfikacji lub przebudowy rdzenia systemu.
- **Niezawodność i tolerancja błędów** — asynchroniczne przetwarzanie komunikatów z wykorzystaniem magistrali powinno gwarantować, że chwilowa niedostępność komponentów nie spowoduje trwałej utraty zlecenia druku ani niespójności stanów w bazie danych.
- **Bezpieczeństwo danych** — autoryzacja dostępu do interfejsów programistycznych REST API realizowana za pomocą kryptograficznie podpisanych tokenów JWT, natomiast hasła użytkowników bezwzględnie utrwalane w bazie danych w postaci kryptograficznych skrótów (ang. *hashes*) wygenerowanych algorytmem *bcrypt*.
- **Responsywność interfejsu** — warstwa prezentacji (aplikacja frontendowa) musi zostać zaprojektowana zgodnie z paradygmatem RWD (ang. *Responsive Web Design*), zapewniając pełną ergonomię i poprawność wyświetlania zarówno na urządzeniach stacjonarnych, jak i mobilnych.
- **Wydajność czasowa** — czas odpowiedzi punktów końcowych API na zapytania synchroniczne nie powinien przekraczać 2 sekund w warunkach standardowego, nominalnego obciążenia infrastruktury.
- **Przenośność i konteneryzacja** — wszystkie komponenty, mikroserwisy oraz bazy danych wchodzące w skład systemu muszą zostać spakowane w niezależne konte-

nery środowiska *Docker*, umożliwiając pełne uruchomienie całego ekosystemu za pomocą jednego polecenia skryptowego (*Docker Compose*).

2.3 Przegląd istniejących rozwiązań

2.3.1 OctoPrint

Oprogramowanie *OctoPrint* [3] stanowi obecnie jedno z najpopularniejszych rozwiązań typu *open-source* dedykowanych do zdalnego zarządzania i monitorowania pojedynczych drukarek 3D. System ten, uruchamiany najczęściej jako usługa serwerowa w środowisku Linux na mikrokomputerze Raspberry Pi, udostępnia responsywny interfejs webowy, programistyczny interfejs REST API oraz rozbudowany ekosystem wtyczek (ang. *plugins*). Umożliwia on bezpośrednie kontrolowanie parametrów mechanicznych i termicznych drukarki, wizualizację kodu sterującego, podgląd wideo z kamery oraz analizę postępu prac.

Głównym ograniczeniem platformy *OctoPrint* jest jednak jej monolityczna architektura, zaprojektowana z myślą o pojedynczym użytkowniku zarządzającym wyłącznie jedną maszyną. Oprogramowanie to nie oferuje natywnych mechanizmów globalnego kolejkowania zadań dla wielu niezależnych kont, zaawansowanego systemu kontroli dostępu opartego na rolach (RBAC) ani integracji z infrastrukturą chmurową. Z tego względu w projekcie *AddiPi* system *OctoPrint* został wykorzystany wyłącznie w charakterze lokalnego, niskopoziomowego sterownika sprzętowego, nad którym nadbudowano rozproszoną warstwę orkiestracji i zarządzania.

2.3.2 Mainsail oraz Fluidt

Ekosystemy *Mainsail* oraz *Fluidt* to nowoczesne, wysoce zoptymalizowane interfejsy webowe współpracujące bezpośrednio z oprogramowaniem układowym (firmware) *Klipper*. Charakteryzują się one niskim narzutem wydajnościowym oraz zaawansowanymi algorytmami kompensacji drgań maszyn. Podobnie jednak jak w przypadku oprogramowania *OctoPrint*, ich architektura została zorientowana na scenariusz jednorodny (jeden operator — jedno urządzenie) i nie przewiduje mechanizmów współdzielenia zasobów sprzętowych ani wielodostępowej orkiestracji kolejki zadań.

2.3.3 Platformy komercyjne

Rozwiązania takie jak *3DprinterOS*, *Polar Cloud* czy *Ultimaker Digital Factory* to komercyjne systemy klasy *fleet management*, przeznaczone do scentralizowanego zarządzania rozproszoną flotą drukarek trójwymiarowych w sektorze przemysłowym oraz edukacyjnym. Platformy te udostępniają zaawansowane moduły analityczne i kolejki zadań, jednak ich implementacja wiąże się z wysokimi, cyklicznymi kosztami subskrypcyjnymi. Dodatkowo są to systemy zamknięte, działające w modelu SaaS (*Software as a Service*), co uniemożliwia pełną kontrolę nad integralnością i lokalizacją przetwarzanych danych — w specyficznych warunkach laboratoriów akademickich stanowi to istotną barierę wdrożeniową.

2.3.4 Wnioski końcowe

Przeprowadzona analiza wykazała, że żadne z dostępnych na rynku rozwiązań o otwartym kodzie źródłowym nie łączy w sobie wielodostępowego systemu uwierzytelniania, asynchronicznego kolejkowania zadań z funkcją planowania kalendarzowego, bezpośredniej integracji z chmurą obliczeniową oraz sterowania bazującego na paradygmacie Internetu Rzeczy (IoT). Projekt *AddiPi* skutecznie wypełnia tę lukę technologiczną. Stanowi on autonomiczne, bezpieczne i wysoce elastyczne środowisko, które pozwala na pełną niezależność infrastrukturalną przy jednoczesnym zachowaniu niskich kosztów utrzymania systemu.

3 Wybór technologii

3.1 Język programowania oraz środowisko uruchomieniowe

3.1.1 TypeScript oraz Node.js

Większość serwisów backendowych oraz aplikację frontendową zaimplementowano w języku TypeScript [9], będącym nadzbiorem języka JavaScript, który wprowadza statyczne typowanie. Wybór ten znacząco ułatwia utrzymanie rozbudowanych baz kodu poprzez wykrywanie błędów na etapie kompilacji, zaawansowane wsparcie środowisk programistycznych (IDE) oraz samodokumentowanie struktury danych za pomocą typów.

Platformę Node.js [11] wybrano ze względu na asynchroniczny, nieblokujący model wejścia-wyjścia. Rozwiązanie to charakteryzuje się wysoką wydajnością w systemach obsługujących liczne, równoległe połączenia HTTP oraz intensywną komunikację z usługami zewnętrznymi, takimi jak bazy danych, kolejki komunikatów czy usługa *Azure IoT Hub*.

3.1.2 Python

Agent działający na platformie Raspberry Pi został zaimplementowany w języku Python [13]. Biblioteki *azure-iot-device* oraz *azure-storage-blob* są w tym środowisku powszechnie dostępne i szczegółowo udokumentowane. Python stanowi standardowe rozwiązanie w przypadku aplikacji uruchamianych na systemach wbudowanych oraz mikrokomputerach jednoukładowych (ang. *Single-Board Computer*, SBC).

3.2 Framework webowy — Express.js

Wszystkie serwisy warstwy Node.js wykorzystują framework Express.js [10] w wersji stabilnej 5.0. Rozwiązanie to charakteryzuje się minimalistyczną architekturą i niskim narzutem wydajnościowym. Framework nie narzuca sztywnej struktury katalogów (ang. *unopinionated framework*), co pozwala na elastyczne dostosowanie architektury każdego z mikroservisów do jego specyficznych wymagań. Express.js zapewnia również ustandaryzowane mechanizmy obsługi żądań pośredniczących (ang. *middleware*), routingu oraz zarządzania błędami.

3.3 Baza danych — Azure Cosmos DB

Azure Cosmos DB [6] to w pełni zarządzana, wielomodelowa baza danych typu NoSQL, oferująca globalną dystrybucję danych, elastyczne partycjonowanie oraz natywną obsługę zróżnicowanych interfejsów programistycznych (SQL, MongoDB, Cassandra, Gremlin, Table). W projekcie *AddiPi* wykorzystano interfejs API SQL (Core), który umożliwia wykonywanie zapytań w standardzie zbliżonym do języka SQL na dokumentach w formacie JSON.

Wybór tej technologii został podyktowany następującymi czynnikami:

- **Model usługowy PaaS (Platform as a Service)** — pełne zarządzanie infrastrukturą przez dostawcę chmurowego eliminuje konieczność administracji serwerem bazy danych,
- **Elastyczny schemat danych** — poszczególne mikroserwisy przechowują obiekty o zróżnicowanej, heterogenicznej strukturze w ramach jednej bazy danych (*addipi*), z podziałem na dedykowane kontenery (*users*, *jobs*, *refresh-tokens*),
- **Natywna integracja** — bezszwowe współdziałanie z pozostałymi komponentami ekosystemu chmurowego *Microsoft Azure*,
- **Automatyczne indeksowanie** — domyślne indeksowanie wszystkich pól w strukturze dokumentu, co optymalizuje czas wykonywania zapytań.

3.4 Repozytorium obiektowe — Azure Blob Storage

Pliki G-code przesyłane przez użytkowników są przechowywane w usłudze repozytorium obiektowego *Azure Blob Storage* [5] w ramach dedykowanego kontenera o nazwie *gc ode*. Usługa ta zapewnia niemal nieograniczoną przestrzeń magazynową, wysoki poziom dostępności, relatywnie niskie koszty utrzymania danych oraz zoptymalizowany interfejs API dedykowany do bezpiecznego przesyłania i pobierania dużych obiektów binarnych (ang. *Binary Large Objects*, BLOB).

3.5 Broker komunikatów — Azure Service Bus

Azure Service Bus [8] to chmurowa usługa kolejkowania komunikatów i zarządzania strumieniami danych klasy korporacyjnej. W architekturze systemu *AddiPi* pełni ona klu-

czową rolę brokera komunikatów (ang. *message broker*) odpowiedzialnego za asynchroniczne i bezprzerwowe pośrednictwo pomiędzy usługą *Files Service* (producentem komunikatów) a komponentem *Queue Service* (konsumentem).

Dedykowana kolejka print-queue zapewnia niezawodną semantykę dostarczania wiadomości typu *at-least-once* (co najmniej raz). Rozwiązanie to gwarantuje, że w przypadku awarii usługi konsumenckiej lub utraty połączenia sieciowego, komunikat nie zostanie skasowany — pozostaje on zabezpieczony w strukturze magistrali chmurowej i może zostać przetworzony powtórnie po przywróceniu operacyjności konsumenta, co bezpośrednio realizuje wymaganie нефункционалне dotyczące tolerancji błędów.

3.6 Sterowanie urządzeniem — Azure IoT Hub

Azure IoT Hub [7] to usługa chmurowa przeznaczona do scentralizowanego zarządzania, bezpiecznego uwierzytelniania oraz monitorowania urządzeń Internetu Rzeczy (IoT). Usługa ta umożliwia pełną dwukierunkową komunikację (ang. *Device-to-Cloud* oraz *Cloud-to-Device*), a także udostępnia mechanizm metod bezpośrednich (ang. *Direct Methods*). Metody te stanowią synchroniczny wzorzec wywołań typu RPC (ang. *Remote Procedure Call*), realizowany na urządzeniach brzegowych komunikujących się za pośrednictwem protokołów MQTT lub AMQP.

W architekturze systemu *AddiPi* usługa *Printer Service* zdalnie wywołuje na agencie (uruchomionym na platformie Raspberry Pi) metody bezpośrednio: `startPrint`, `cancelPrint` oraz `getStatus`. W odpowiedzi zwrotnej agent transmituje asynchroniczny strumień danych telemetrycznych w postaci zdarzeń systemowych (m.in. `print_started`, `print_progress` czy `print_completed`). Komunikaty te są agregowane przez usługę *Printer Service* za pośrednictwem punktu końcowego zgodnego ze specyfikacją *Azure Event Hubs*.

3.7 Interfejs użytkownika — React oraz Vite

Warstwa prezentacji (interfejs użytkownika) została zaimplementowana z wykorzystaniem biblioteki *React* [4] w wersji 18 oraz języka TypeScript. W charakterze środowiska deweloperskiego oraz narzędzia do automatyzacji procesu budowania (ang. *bundler*) aplikacji wykorzystano framework *Vite* [15], co pozwoliło na optymalizację czasu kompilacji kodu źródłowego.

Do zarządzania globalnym stanem aplikacji klienckiej zastosowano bibliotekę *Zustand* [12]. Rozwiązanie to umożliwia wydajne współdzielenie danych pomiędzy komponentami drzewa DOM w sposób bezstanowy, eliminując konieczność implementowania nadmiarowej i złożonej architektury (takiej jak *Redux*) przy jednoczesnym zachowaniu wysokiej reaktywności interfejsu.

Projekt graficzny oraz warstwa stylów zostały zrealizowane przy użyciu frameworka *Tailwind CSS* [14]. Narzędzie to bazuje na architekturze klas narzędziowych (ang. *utility-first*), w której style interfejsu są komponowane bezpośrednio w strukturze kodu za pomocą predefiniowanych klas reprezentujących pojedyncze właściwości kaskadowych arkuszy stylów.

3.8 Konteneryzacja — Docker oraz Docker Compose

W celu zapewnienia powtarzalności środowiska uruchomieniowego, każdy z mikroserwisów wchodzących w skład ekosystemu posiada dedykowany plik konfiguracyjny *Dockerfile*. Proces budowania obrazów realizuje mechanizm wieloetapowy (ang. *multi-stage build*). Podejście to pozwala na drastyczne ograniczenie rozmiaru finalnych obrazów kontenerów oraz całkowite odizolowanie środowiska kompilacji i zależności deweloperskich od produkcyjnego środowiska uruchomieniowego.

Narzędzie *Docker Compose* [2] wykorzystano do zautomatyzowanej orkiestracji i zarządzania cyklem życia wszystkich kontenerów zarówno w środowisku lokalnym, jak i produkcyjnym. Narzędzie to definiuje odizolowaną, wspólną sieć wirtualną *addip-network* oraz odpowiada za bezpieczne wstrzykiwanie zmiennych środowiskowych odczytywanych z pliku *.env*.

3.9 CAD - SolidWorks

Projekt mechaniczny obudowy urządzenia wykonano w programie **SolidWorks** [1], stanowiącym środowisko do parametrycznego modelowania bryłowego oraz projektowania złożów mechanicznych. Oprogramowanie umożliwia tworzenie asocjatywnych modeli 3D, w których modyfikacja parametrów części powoduje automatyczną aktualizację złożów oraz dokumentacji technicznej.

Wybór programu SolidWorks był podyktowany dostępnością oprogramowania w ramach licencji studenckiej oraz szerokim wsparciem dla formatów eksportu modeli, ta-

kich jak .STEP (neutralny format wymiany modeli CAD), .STL (format wykorzystywany w procesie przygotowania modeli o druku 3D) oraz .3MF (format wymiany zawierający dodatkowe metadane procesu druku).

Projekt został podzielony na pliki .SLDPRT, reprezentujące pojedyncze części, oraz .SLDASM, definiujące kompletne złożenia mechaniczne.

3.10 Porównanie wybranych alternatyw

Proces projektowania systemu wymagał wielokryterialnej analizy dostępnych rozwiązań technologicznych. Decyzje architektoniczne oparto na kryteriach takich jak: stopień integracji z chmurą obliczeniową, łatwość skalowania, nakłady na zarządzanie infrastrukturą oraz spójność ekosystemu programistycznego. Zbiorcze zestawienie wszystkich podjętych decyzji projektowych, wybranych technologii oraz odrzuconych alternatyw przedstawiono na rysunku 1.

Kategoria	Wybrana technologia	Odrzucone alternatywy	Kluczowy powód
Język backendowy serwisy Node.js + agent	TypeScript / Node.js + Python (agent RPI) Express.js v5	Go, Java, PHP	statyczne typowanie, duży ekosystem npm, nieblokujące I/O
Baza danych users, jobs, tokens	Azure Cosmos DB SQL API (JSON) PaaS, bez serwera	PostgreSQL, MongoDB Atlas	natywna integracja z Azure, elastyczny schemat JSON
Kolejka komunikatów upload → zadanie	Azure Service Bus kolejka print-queue at-least-once, PaaS	RabbitMQ, Apache Kafka	brak zarządzania serwerem, auto- skalowanie
Sterowanie IoT chmura → Raspberry Pi	Azure IoT Hub Direct Methods, MQTT telemetry D2C	AWS IoT Core, MQTT broker własny	Direct Methods, ekosystem Azure, telemetry wbudowana
Frontend aplikacja webowa	React 18 + Vite TypeScript, Zustand Tailwind CSS, i18next	Vue, Angular, Next.js	dojrzały ekosystem, hooks, SPA bez SSR (prostota)
Wdrożenie konteneryzacja	Docker + Compose multi-stage build addip-network	Kubernetes, Podman, bare metal	przenośność, jedna komenda uruchomienia, izolacja serwisów

Rysunek 1: Zbiorcze zestawienie wybranej architektury technologicznej, odrzuconych alternatyw oraz kluczowych powodów decyzyjnych.

Głównym założeniem przy wyborze warstwy backendowej oraz frontendowej było

ujednolicenie stosu technologicznego wokół języka TypeScript oraz środowiska Node.js. Pozwoliło to na współdzielenie typów danych między aplikacją webową (React 18 + Vite) a usługami serwerowymi, co znacząco przyspieszyło proces implementacji. Wyjątek stanowi mikrousluga *Files Service* oraz komponent agenta uruchamiany na platformie Raspberry Pi, gdzie ze względu na specyfikę obsługi interfejsów sprzętowych zdecydowano się na wykorzystanie języka Python.

Kluczowe znaczenie dla stabilności systemu miało precyzyjne dobranie silnika bazy danych oraz mechanizmu wymiany komunikatów. Szczegółowe porównanie cech rozważanych systemów bazodanowych oraz systemów kolejkowych zestawiono odpowiednio w tabeli 1 oraz tabeli 2.

Tabela 1: Porównanie baz danych rozważanych w projekcie

Cecha	Cosmos DB	PostgreSQL	MongoDB Atlas
Model danych	Dokument (JSON)	Relacyjny	Dokument (BSON)
Zarządzanie	PaaS (bez serwera)	Samodzielny/PaaS	PaaS
Integracja z Azure	Natywna	Ograniczona	Pośrednia
Elastyczny schemat danych	Tak	Nie	Tak
Dostępność bezpłatnego planu	Tak (400 RU/s)	Tak (self-hosted)	Tak (512 MB)

Jak wynika z tabeli 1, baza Azure Cosmos DB została wybrana ze względu na bezobsługowy model bezserwerowy (Serverless PaaS) oraz natywną integrację z ekosystemem Azure. Elastyczny schemat JSON idealnie odpowiada dynamicznej strukturze danych generowanych przez sensory IoT, eliminując narzut związany z migracjami schematu w tradycyjnych bazach relacyjnych (takich jak PostgreSQL).

Tabela 2: Porównanie rozwiązań kolejkowania wiadomości

Cecha	Azure Service Bus	RabbitMQ	Apache Kafka
Model wdrożenia	PaaS	Self-hosted	Self-hosted / PaaS
Semantyka dostarczania	At-least-once	At-least-once	At-least-once
Złożoność konfiguracji	Niska	Średnia	Wysoka
Integracja z Azure	Natywna	Ograniczona	Pośrednia
Skalowalność	Automatyczna	Ręczna	Wysoka

W przypadku infrastruktury kolejki (tab. 2), usługa Azure Service Bus przewyższyła alternatywy brakiem konieczności samodzielnego zarządzania serwerem oraz wbudowaną automatyczną skalowalnością. Narzut konfiguracyjny rozwiązań takich jak Apache

Kafka okazał się nieuzasadniony w stosunku do skali projektu, natomiast Azure Service Bus zapewnił wymaganą semantykę dostarczania wiadomości *at-least-once* bezpośrednio po uruchomieniu usługi.

Ostatnim etapem konfiguracji było zdefiniowanie środowiska uruchomieniowego. Odrzucono złożone systemy orkiestracji (Kubernetes) oraz instalacje bezpośrednie (bare metal) na rzecz narzędzia Docker + Compose. Zapewniło to pełną przenośność kodu, powtarzalność wdrożeń przy użyciu wieloetapowego budowania obrazów (*multi-stage build*) oraz bezpieczną izolację usług w ramach dedykowanej sieci wirtualnej o nazwie `addip-network`.

4 Architektura systemu

4.1 Ogólna struktura

System AddiPi zbudowany jest w architekturze mikroserwisowej. Każdy serwis stanowi niezależną jednostkę wdrożeniową odpowiedzialną za określoną domenę biznesową oraz komunikuje się z pozostałymi komponentami za pośrednictwem zdefiniowanych interfejsów (HTTP REST lub kolejka komunikatów).

W ramach systemu wyodrębniono następujące komponenty:

1. **AddiPi Auth Service** (port 3001) - realizuje proces uwierzytelniania użytkowników oraz obsługę tokenów JWT.
2. **AddiPi User Service** (port 3002) - odpowiada za zarządzanie profilami użytkowników oraz historią zadań do wydruku.
3. **AddiPi Files Service** (port 5000) - umożliwia przesyłanie oraz przechowywanie plików G-code.
4. **AddiPi Queue Service** (porty 3070) - odbiera komunikaty z kolejki oraz zapisuje informacje o zadaniach w bazie Azure Cosmos DB.
5. **AddiPi Printer Service** (port 3050) - odpowiada za harmonogramowanie zadań do wydruku, komunikację z Azure IoT Hub oraz odbiór telemetrii urządzenia.
6. **AddiPi Video Service** (port 3003) - odpowiada za rejestrację i udostępnianie adresu URL kamery systemu OctoPrint.
7. **AddiPi Agent** (Raspberry Pi) - agent uruchamiany na urządzeniu Raspberry Pi, pośredniczący pomiędzy Azure IoT Hub a systemem OctoPrint.
8. **AddiPi Frontend** (Vercel / port 5173) - aplikacja webowa zbudowana z wykorzystaniem biblioteki React.

4.2 Diagram architektury

Na rysunku 2 przedstawiono sekwencyjny przepływ danych pomiędzy poszczególnymi komponentami zaprojektowanego systemu.

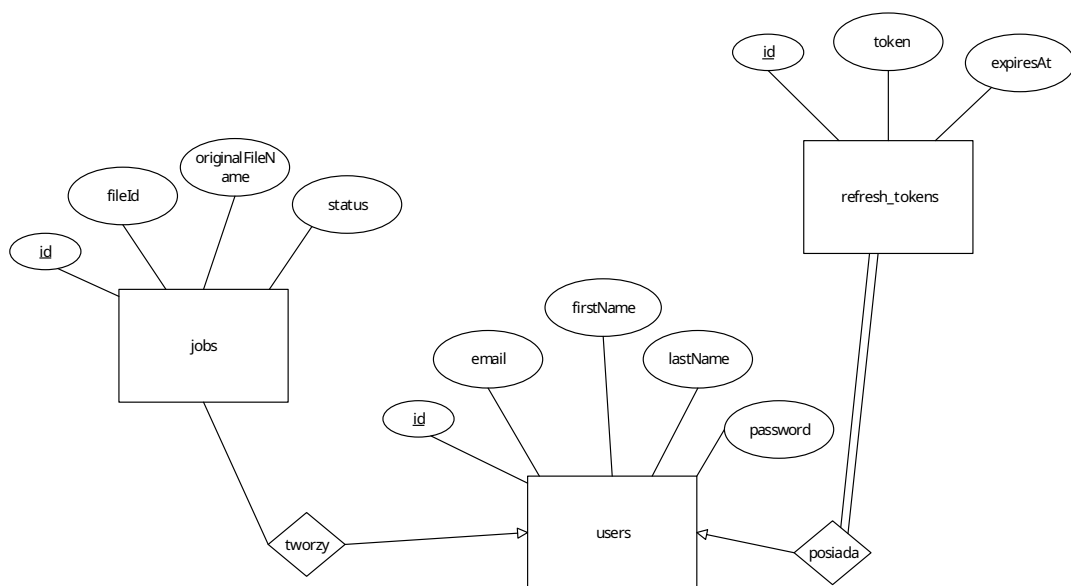
oczekujących na realizację. Jeżeli urządzenie drukujące jest wolne, a zadanie spełnia kryteria rozpoczęcia procesu wytwórczego, status transakcji jest zmieniany na `waiting_for_printer_ready`. System przechodzi wówczas w stan oczekiwania na potwierdzenie gotowości operacyjnej drukarki.

5. **Inicjacja komunikacji IoT:** Po autoryzacji gotowości urządzenia przez użytkownika lub administratora, usługa *Printer Service* zdalnie wywołuje metodę bezpośrednią (ang. *Direct Method*) `startPrint` na dedykowanym agencie, wykorzystując do tego celu usługę *Azure IoT Hub*.
6. **Zarządzanie procesem drukowania przez agenta:** Komponent *AddiPi Agent* pobiera przypisany plik G-code z repozytorium *Azure Blob Storage*, przekazuje go bezpośrednio do interfejsu oprogramowania *OctoPrint* oraz inicjuje proces drukowania 3D. W trakcie pracy agent w sposób ciągły transmituje dane telemetryczne do usługi *Azure IoT Hub*, raportując bieżący postęp, status ukończenia lub ewentualne błędy krytyczne.
7. **Odbiór telemetrii i aktualizacja stanu:** Usługa *Printer Service* konsumuje strumień danych telemetrycznych za pośrednictwem punktu końcowego zgodnego ze specyfikacją *Azure Event Hubs*, a następnie aktualizuje stan procesu w bazie *Azure Cosmos DB*.
8. **Odświeżanie danych w interfejsie:** Aplikacja kliencka (frontendowa) realizuje cykliczne zapytania sondujące (ang. *polling* z interwałem 5 sekund) do mikroserwisów, zapewniając bieżącą aktualizację stanu systemu prezentowanego użytkownikowi.

4.3 Model danych

W bazie danych *Azure Cosmos DB* dane są strukturyzowane i przechowywane w postaci dokumentów w formacie JSON. Zostały one pogrupowane w logicznych kontenerach, odpowiadających poszczególnym domenom biznesowym systemu.

Na rysunku 3 przedstawiono diagram encji i relacji (ang. *Entity-Relationship Diagram*, ERD) zaprojektowanej bazy danych *addipi-cosmos*, sporządzony w klasycznej notacji Chena. Diagram ten odwzorowuje logiczne powiązania referencyjne pomiędzy dokumentami w poszczególnych kontenerach.



Rysunek 3: Diagram bazy danych addipi-cosmos w notacji Chena.

4.3.1 Użytkownik (kontener users)

Dokument przechowuje informacje profilowe użytkowników, dane autoryzacyjne oraz status weryfikacji konta. Przykładową strukturę przedstawiono na listingu 1.

```

1  {
2    "id": "user_<timestamp>",
3    "email": "student@uwr.edu.pl",
4    "firstName": "Jan",
5    "lastName": "Kowalski",
6    "password": "<brypt hash>",
7    "role": "user",
8    "isVerified": true,
9    "verificationToken": "<hex>",
10   "verificationTokenExpiry": "2025-01-01T00:00:00Z",
11   "createdAt": "2025-01-01T00:00:00Z",
12   "updatedAt": "2025-01-01T00:00:00Z"
13 }
  
```

Listing 1: Przykładowy dokument użytkownika w Azure Cosmos DB

Pole `role` definiuje uprawnienia w systemie i może przyjmować wartość `user` (użytkownik standardowy) lub `admin` (administrator systemu).

4.3.2 Zadanie do wydruku (kontener `jobs`)

Dokument zadania (listing 2) agreguje informacje o pliku sterującym, powiązonym użytkowniku, metrykach czasu oraz bieżącym stanie realizacji procesu produkcyjnego.

```
1  {
2    "id": "<timestamp string>",
3    "fileId": "20251203_014648_model.gcode",
4    "originalFileName": "model.gcode",
5    "userId": "user_<timestamp>",
6    "userEmail": "student@uwr.edu.pl",
7    "status": "printing",
8    "scheduledAt": "2025-12-01T15:00:00Z",
9    "createdAt": "2025-11-30T10:00:00Z",
10   "startedAt": "2025-12-01T15:01:00Z",
11   "completedAt": "2025-12-01T17:30:00Z",
12   "progress": 87.5,
13   "printTime": 4500,
14   "printTimeLeft": 600,
15   "deviceId": "raspberrypi-mkt-01",
16   "failureReason": null
17 }
```

Listing 2: Przykładowy dokument zadania do wydruku w Cosmos DB

Pole `status` może przyjmować wartości: `pending`, `scheduled`, `waiting_for_printer_ready`, `delayed`, `printing`, `completed`, `failed`, `cancelled`.

4.3.3 Token odświeżania (kontener `refresh-tokens`)

W celu zapewnienia bezpiecznej, bezstanowej autoryzacji sesji, tokeny odświeżania (ang. *refresh tokens*) są utrwalane w dedykowanym kontenerze (listing 3).

```
1  {
2    "id": "<userId>_<timestamp>",
3    "userId": "user_<timestamp>",
4    "token": "<JWT string>",
5    "expiresAt": "2025-12-08T00:00:00Z",
```

```

6   "createdAt": "2025-12-01T00:00:00Z"
7 }

```

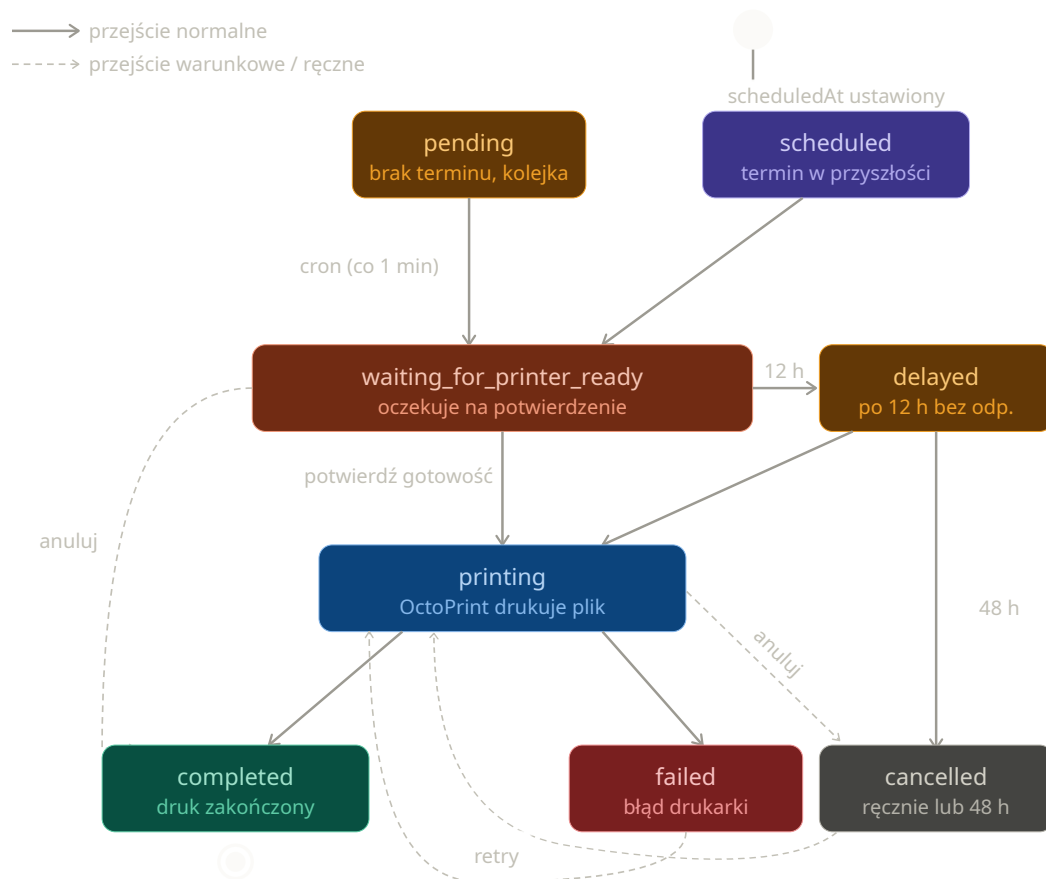
Listing 3: Przykładowy dokument refresh tokena

4.4 Cykl życia zadania do wydruku

Zadanie w procesie przetwarzania i realizacji może znajdować się w jednym z następujących stanów:

- **pending** — zadanie bez określonego terminu realizacji, oczekujące w kolejce ogólnej systemu.
- **scheduled** — zadanie posiadające ściśle zaplanowany termin realizacji, oczekujące na osiągnięcie wskazanego czasu rozpoczęcia. Po nadejściu zaplanowanego czasu zadanie to otrzymuje najwyższy priorytet wykonania.
- **waiting_for_printer_ready** — usługa harmonogramująca (ang. *scheduler*) wybrała zadanie do realizacji; system oczekuje na manualne potwierdzenie gotowości urządzenia przez użytkownika.
- **delayed** — stan opóźnienia, w którym system oczekuje na potwierdzenie gotowości drukarki dłużej niż 3 h; do użytkownika wysyłane jest automatyczne powiadomienie przypominające.
- **printing** — proces drukowania modelu sterującego jest aktywnie realizowany przez oprogramowanie *OctoPrint*.
- **completed** — proces drukowania 3D został pomyślnie sfinalizowany.
- **failed** — w trakcie realizacji procesu produkcyjnego wystąpił błąd krytyczny uniemożliwiający dalsze drukowanie.
- **cancelled** — zadanie zostało anulowane ręcznie przez użytkownika bądź administratora lub automatycznie przez system po upływie 48 h oczekiwania na potwierdzenie gotowości urządzenia.

Na rysunku 4 przedstawiono formalny model maszyny stanów, obrazujący dopuszczalne przejścia fazowe w cyklu życia zadania produkcyjnego.



Rysunek 4: Diagram maszyny stanów cyklu życia zadania do wydruku.

4.5 Bezpieczeństwo i uwierzytelnianie

W celu zapewnienia poufności danych oraz kontroli dostępu, w systemie zaimplementowano dwutokenowy model uwierzytelniania oparty na standardzie JWT (*JSON Web Token*). Mechanizm ten wykorzystuje dwa rodzaje obiektów:

- **Access token** — token dostępu o krótkim czasie autoryzacji (ważny przez 15 minut), przekazywany w nagłówku protokołu HTTP (`Authorization: Bearer <token>`). Integralność i autentyczność tokenu są weryfikowane kryptograficznie przy użyciu klucza symetrycznego `JWT_SECRET`.
- **Refresh token** — token odświeżający o wydłużonym czasie ważności (wynoszącym 7 dni), utrwalany w bazie danych *Azure Cosmos DB* oraz przechowywany po stronie aplikacji klienckiej. Podpisywany jest przy użyciu dedykowanego klucza `JWT_REFRESH_SECRET`. Służy do generowania nowej pary tokenów bez konieczności ponownego wprowadzania danych uwierzytelniających przez użytkownika.

Hasła użytkowników nie są przechowywane w bazie danych w postaci jawnej. Do ich zabezpieczenia wykorzystano algorytm funkcjonału skrótu *bcrypt* z parametrem kosztu (ang. *salt rounds*) ustawionym na wartość 10.

Proces rejestracji nowych kont został ze względów bezpieczeństwa ograniczony wyłącznie do adresów poczty elektronicznej w domenie akademickiej @uwr.edu.pl. Pełna aktywacja profilu wymaga weryfikacji dwuetapowej poprzez kliknięcie w unikalny link aktywacyjny przesłany na adres e-mail użytkownika.

Kontrola dostępu na poziomie kodu realizowana jest za pomocą oprogramowania pośredniczącego (ang. *middleware*):

- `requireAuth` — odpowiada za ekstrakcję i weryfikację tokenu dostępu poprzez wysłanie wewnętrznego żądania weryfikacyjnego do punktu końcowego POST `/auth/verify` usługi *Auth Service*.
- `requireAdmin` — realizuje mechanizm kontroli dostępu opartej na rolach (ang. *Role-Based Access Control*, RBAC), sprawdzając, czy zdekodowany profil użytkownika posiada przypisane uprawnienia klasy admin.

5 Implementacja serwisów backendowych

5.1 Auth Service

5.1.1 Opis ogólny

Usługa *Auth Service* odpowiada za procesy uwierzytelniania, autoryzacji oraz zarządzania kontami użytkowników. W zakres jej funkcjonalności wchodzi: rejestracja, logowanie i wylogowanie użytkowników, odświeżanie tokenów sesyjnych oraz weryfikacja adresów poczty elektronicznej.

Komponent ten został zaimplementowany w środowisku uruchomieniowym Node.js z wykorzystaniem frameworka Express.js oraz języka TypeScript. Serwis realizuje operacje wejścia-wyjścia poprzez komunikację z bazą danych *Azure Cosmos DB* (w ramach kontenerów *users* oraz *refresh-tokens*). Ponadto integruje się z zewnętrznym serwerem SMTP firmy Google w celu dystrybucji wiadomości aktywacyjnych, wykorzystując do tego celu bibliotekę Nodemailer.

5.1.2 Punkty końcowe API

W tabeli 3 zestawiono i opisano punkty końcowe (ang. *endpoints*) udostępniane przez usługę *Auth Service*.

5.1.3 Kluczowe fragmenty implementacji

Na listingu 4 przedstawiono implementację funkcji odpowiedzialnych za generowanie tokenu dostępu oraz tokenu odświeżającego przy użyciu biblioteki *jsonwebtoken*.

```
1 export function generateAccessToken(user: User): string {
2     const payload: JWTPayload = {
3         userId: user.id,
4         email: user.email,
5         role: user.role
6     };
7     return jwt.sign(payload, CONFIG.JWT_SECRET, { expiresIn: '15m'
8         });
9 }
10 export function generateRefreshToken(user: User): string {
```

Tabela 3: Punkty końcowe usługi Auth Service.

Metoda	Ścieżka	Opis
POST	/auth/register	Rejestracja nowego użytkownika, walidacja domeny e-mail, generowanie szyfru hasła oraz wysłanie wiadomości z linkiem aktywacyjnym.
POST	/auth/login	Uwierzytelnienie użytkownika; generowanie i zwrot pary tokenów: <i>access token</i> oraz <i>refresh token</i> .
POST	/auth/logout	Unieważnienie tokenu odświeżającego i zakończenie aktywnej sesji użytkownika.
PATCH	/auth/refresh	Generowanie nowego tokenu dostępu na podstawie ważnego tokenu odświeżającego.
POST	/auth/verify	Weryfikacja poprawności tokenu dostępu, wykorzystywana wewnętrznie przez pozostałe mikroserwisy.
GET	/auth/verify-email	Aktywacja konta użytkownika po odebraniu i weryfikacji tokenu z linku aktywacyjnego.
POST	/auth/resent-verification	Ponowne wygenerowanie i wysłanie wiadomości weryfikacyjnej na adres e-mail.
GET	/health	Punkt końcowy diagnostyki stanu działania usługi (ang. <i>health check</i>).

```

11 return jwt.sign(
12   { userId: user.id },
13   CONFIG.JWT_REFRESH_SECRET,
14   { expiresIn: '7d' }
15 );
16 }

```

Listing 4: Implementacja generowania tokenów JWT (token-service.ts)

Proces walidacji tokenu odświeżającego (ang. *refresh token*) jest dwuetapowy. Obejmuje zarówno kryptograficzną weryfikację poprawności podpisu cyfrowego struktury JWT, jak i asynchroniczne sprawdzenie obecności oraz ważności powiązanego rekordu w bazie danych *Azure Cosmos DB*. Takie podejście architektoniczne umożliwia natychmiastowe, zdalne unieważnianie sesji (ang. *token revocation*) w momencie wylogowania się użytkownika z systemu. Kod realizujący tę procedurę zaprezentowano na listingu 5.

```

1 export async function validateRefreshToken(
2   token: string
3 ): Promise<string | null> {
4   try {
5     const decoded = jwt.verify(
6       token,
7       CONFIG.JWT_REFRESH_SECRET
8     ) as { userId: string };
9
10    const query = `SELECT * FROM c
11      WHERE c.token = @token
12      AND c.userId = @userId
13      AND c.expiresAt > @now`;
14
15    const { resources } = await refreshTokensContainer.items.query(
16      {
17        query,
18        parameters: [
19          { name: '@token', value: token },
20          { name: '@userId', value: decoded.userId },
21          { name: '@now', value: new Date().toISOString() }
22        ]
23      }).fetchAll();

```

```

24     return resources.length > 0 ? decoded.userId : null;
25   } catch {
26     return null;
27   }
28 }

```

Listing 5: Implementacja walidacji refresh tokenu (token-service.ts)

5.1.4 Weryfikacja adresu poczty elektronicznej

Po pomyślnym zakończeniu procesu rejestracji system generuje unikalny, 64-znakowy token w formacie szesnastkowym. Jest on tworzony przy użyciu kryptograficznie bezpiecznego generatora liczb pseudolosowych z modułu `crypto` (`crypto.randomBytes`). Zapewnia to wysoką entropię i odporność na ataki typu *brute-force*.

Wygenerowany identyfikator jest zapisywany w bazie danych wraz z datą ważności, a do użytkownika wysyłana jest wiadomość e-mail zawierająca odsyłacz aktywacyjny skierowany do punktu końcowego systemu:

```
GET /auth/verify-email?token=<hex>
```

Po odebraniu żądania przez usługę *Auth Service*, system przeprowadza weryfikację poprawności oraz ważności tokenu. W przypadku pozytywnego rezultatu walidacji, atrybut `isVerified` w dokumencie użytkownika przyjmuje wartość `true`, a konto zostaje w pełni aktywowane. Na zakończenie procesu serwer wykonuje przekierowanie (ang. *HTTP redirect*) przeglądarki użytkownika do adresu bazowego aplikacji klienckiej.

5.2 User Service

5.2.1 Opis ogólny

Usługa *User Service* realizuje zadania związane z zarządzaniem profilami użytkowników, modyfikacją ich uprawnień oraz agregacją danych dotyczących powiązanych zadań wytwórczych. Komponent ten wykonuje operacje odczytu i zapisu danych w ramach kontenerów `users` oraz `jobs` w bazie danych *Azure Cosmos DB*.

Autoryzacja żądań oraz weryfikacja tożsamości użytkowników są realizowane w sposób scentralizowany poprzez przesyłanie wewnętrznych zapytań do punktu końcowego `POST /auth/verify` opisanej wcześniej usługi *Auth Service*.

5.2.2 Punkty końcowe API

W tabeli 4 wyszczególniono i opisano punkty końcowe (ang. *endpoints*) udostępniane przez usługę *User Service*, z uwzględnieniem podziału na operacje standardowe oraz administracyjne.

Tabela 4: Punkty końcowe usługi User Service.

Metoda	Ścieżka	Opis
GET	/users/me	Pobranie danych profilowych zalogowanego użytkownika.
PATCH	/users/me	Aktualizacja danych osobowych użytkownika (imię oraz nazwisko).
GET	/users/me/jobs	Pobranie historii wszystkich zadań przypisanych do zalogowanego użytkownika.
GET	/users/me/stats	Pobranie statystyk użytkownika, obejmujących m.in. zagregowaną liczbę zadań z podziałem na ich statusy.
GET	/users/jobs/upcoming	Pobranie listy zadań oczekujących na realizację (posiadających status pending lub scheduled).
GET	/users/jobs/recent-completed	Pobranie historycznej listy ostatnio sfinalizowanych procesów druku.
GET	/users	Pobranie rejestru wszystkich użytkowników systemu (dostęp zastrzeżony dla administratora).

Tabela 4 – ciąg dalszy z poprzedniej strony

Metoda	Ścieżka	Opis
PATCH	/users/:id/role	Modyfikacja roli i poziomu uprawnień użytkownika o wskazanym identyfikatorze (dostęp zastrzeżony dla administratora).
DELETE	/users/:id	Trwałe usunięcie konta użytkownika na podstawie parametru identyfikatora (dostęp zastrzeżony dla administratora).
GET	/users/:id/jobs	Pobranie pełnej listy zadań powiązanych z wybranym kontem użytkownika (dostęp zastrzeżony dla administratora).

5.2.3 Mechanizm autoryzacji żądań

Na listingu 6 zaprezentowano produkcyjną implementację oprogramowania pośredniczącego `requireAuth`. Komponent ten w pełni wykorzystuje silne typowanie języka TypeScript, w tym parametry generyczne interfejsów `Request` oraz `Response` z biblioteki `express`.

Procedura autoryzacji polega na bezpiecznej ekstrakcji tokenu dostępu z nagłówka `Authorization`, a następnie wykonaniu asynchronicznego żądania sieciowego metodą `POST` do punktu końcowego usługi *Auth Service*. Po odebraniu odpowiedzi struktura danych jest rzutowana na dedykowany typ `Data`. W przypadku poprawnej walidacji następuje asercja typu obiektu żądania, wstrzyknięcie profilu użytkownika do pola `user` oraz wywołanie funkcji `next()`, która przekazuje sterowanie do kolejnego elementu w potoku przetwarzania (ang. *middleware chain*).

```

1 import { CONFIG } from "../config/config";
2 import type { Request, Response, NextFunction } from "express";
3 import { Data, AuthUser } from "../mwTypes";
4

```

```

5  const requireAuth = async (
6  req: Request<{}>, unknown, {}, {}>,
7  res: Response<{error: string}>,
8  next: NextFunction
9  ): Promise<void | {error: string}> => {
10     try {
11         const token: string | undefined = req.headers.authorization?.
            replace('Bearer ', '');
12
13         if (!token) {
14             res.status(401).json({ error: 'Missing authentication token'
15             });
16             return;
17         }
18
19         const response: globalThis.Response = await fetch(`${CONFIG.
20             AUTH_SERVICE_URL}/auth/verify`, {
21             method: 'POST',
22             headers: {
23                 'Authorization': `Bearer ${token}`,
24                 'Content-Type': 'application/json'
25             }
26         });
27
28         const data: Data = await response.json() as Data;
29
30         if (!response.ok) {
31             throw new Error('Authentication failed');
32         }
33
34         (req as any).user = data.user;
35
36         next();
37     } catch (err) {
38         res.status(401).json({ error: 'Invalid authentication token'
39         });

```

```
40 export default requireAuth;
```

Listing 6: Implementacja oprogramowania pośredniczącego `requireAuth` (`requireAuth.ts`)

5.3 Files Service

5.3.1 Opis ogólny

Usługa *Files Service* stanowi mikroservis zaimplementowany w języku Python z wykorzystaniem mikroframeworka Flask. Komponent ten odpowiada za asynchroniczne przyjmowanie plików z kodem sterującym (G-code) od aplikacji klienckiej, ich bezpieczne utrwalanie w repozytorium obiektowym *Azure Blob Storage* oraz rozgłaszanie zdarzeń systemowych za pośrednictwem magistrali komunikatów *Azure Service Bus*.

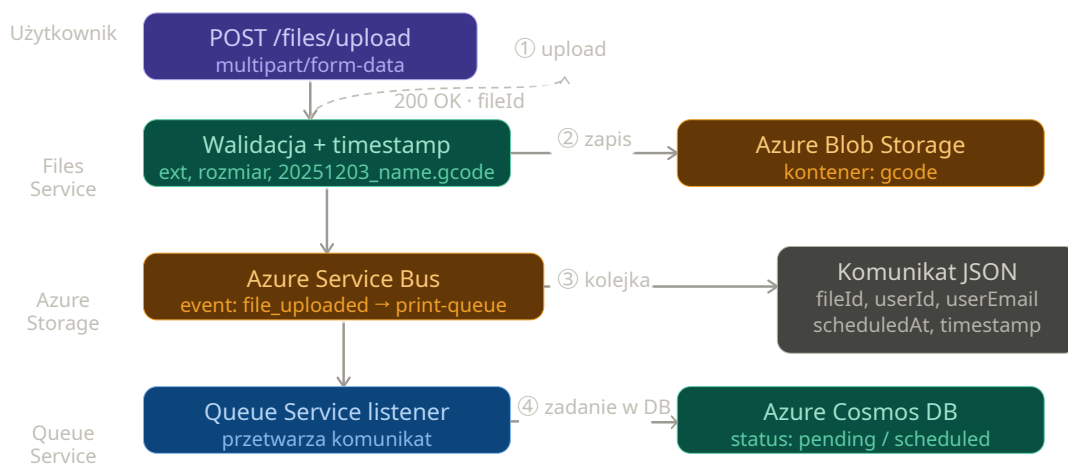
5.3.2 Procedura przetwarzania i przesyłania plików

Przebieg procesu przesyłania pliku oraz jego rejestracji w systemie realizowany jest sekwencyjnie według następującego scenariusza:

1. **Inicjacja żądania:** Aplikacja kliencka wysyła żądanie `POST /files/upload` zawierające plik zakodowany w formacie `multipart/form-data`.
2. **Weryfikacja uprawnień:** Oprogramowanie pośredniczące `require_auth` deleguje proces walidacji tokenu uwierzytelniającego do usługi *Auth Service*.
3. **Walidacja potoku wejściowego:** System weryfikuje poprawność formatu pliku (akceptowane jest wyłącznie rozszerzenie `.gcode`), jego rozmiar (wprowadzono limit do 50 MB) oraz opcjonalnie strukturę wewnętrzną (w przypadku aktywacji flagi konfiguracyjnej `STRICT_CONTENT_CHECK`).
4. **Sanitaryzacja i unikalność nazw:** Nazwa pliku wejściowego jest modyfikowana poprzez dodanie prefiksu czasowego zawierającego kompletną datę i godzinę odebrania żądania (np. `20251203_014648_model.gcode`), co zapobiega konfliktom nadpisywania zasobów.
5. **Utrwalenie zasobu:** Plik w formacie binarnym zostaje przesłany i zapisany w dedykowanym kontenerze `gcode` w ramach usługi *Azure Blob Storage*.

6. **Publikacja zdarzenia:** Do kolejki print-queue magistrali *Azure Service Bus* wysyłany jest komunikat w formacie JSON, który inicjuje dalsze etapy cyklu życia zadania.

Na rysunku 5 zobrazowano schematyczny przepływ danych podczas procedury przesyłania i kolejkowania pliku G-code.



Rysunek 5: Przepływ przesyłania pliku G-code — od zapytania HTTP do integracji z kolejką.

Na listingu 7 zaprezentowano implementację końcowego etapu obsługi żądania w ramach kontrolera usługi *Files Service*. Po pomyślnej walidacji i zapisie pliku w repozytorium obiektowym, struktura danych reprezentująca zdarzenie jest serializowana do formatu JSON i przekazywana do metody nadawczej klienta *Azure Service Bus*. Całość operacji została otoczona blokiem obsługi wyjątków (`try-except`), co gwarantuje stabilność działania mikroservisu i zwrócenie ustrukturyzowanej odpowiedzi o błędzie w formacie JSON wraz ze statusem HTTP 500 w przypadku awarii infrastruktury chmuwej.

```
1 message = {
2     'event': 'file_uploaded',
3     'fileId': filename,
4     'originalFileName': original_filename,
5     'userId': user_id,
6     'userEmail': user_email,
```

```

7         'timestamp': str(time.time()),
8         'scheduledAt': request.form.get('scheduledAt')
9     }
10
11     sb_client = current_app.config.get('SERVICE_BUS_CLIENT')
12     if sb_client:
13         with sb_client:
14             sender = sb_client.get_queue_sender(queue_name='print-
15             queue')
16             sender.send_messages([{'body': json.dumps(message)}])
17
18         return jsonify({'status': 'success', 'fileId': filename}), 200
19
20 except Exception as e:
21     return jsonify({'error': str(e)}), 500

```

Listing 7: Implementacja publikacji komunikatu w Azure Service Bus (files_controller.py)

5.4 Queue Service

5.4.1 Opis ogólny

Usługa *Queue Service* to mikroservis zaimplementowany w środowisku uruchomieniowym Node.js przy użyciu języka TypeScript oraz standardu modułów ECMAScript (ang. *ECMAScript Modules*, ESM). Głównym zadaniem komponentu jest asynchroniczne i bezprzerwowe nasłuchiwanie (ang. *listening*) komunikatów publikowanych w magistrali *Azure Service Bus*, a następnie ich transformacja i utrwalanie w postaci dokumentów zadań produkcyjnych w bazie danych *Azure Cosmos DB*.

Dodatkowo serwis udostępnia dedykowany interfejs programistyczny HTTP API, który umożliwia innym komponentom systemu (w tym aplikacji klienckiej) monitorowanie stanu kolejki oraz zarządzanie priorytetami zadań.

5.4.2 Mechanizm nasłuchiwania kolejki

Na listingu 8 przedstawiono implementację procedury odpowiedzialnej za rejestrację i obsługę potoku zdarzeń pochodzących z kolejki chmurowej. Logika biznesowa realizuje

bezpieczne parsowanie komunikatów w formacie JSON oraz obsługuje scenariusz awaryjny (ang. *dead-lettering/abandonment*) poprzez wywołanie metody `abandonMessage()`, co pozwala na zachowanie spójności danych w przypadku wystąpienia błędów infrastrukturalnych.

```
1 export function startPrintQueueListener(container, receiver) {
2   if (!container) throw new Error('container is required for
   printQueueListener');
3   if (!receiver) throw new Error('receiver is required for
   printQueueListener');
4
5   const messageHandler = async (message) => {
6     try {
7       let data = message.body;
8       if (typeof data === 'string') {
9         try {
10          data = JSON.parse(data);
11        } catch(e) {
12          await receiver.abandonMessage(message);
13          return;
14        }
15      }
16
17      if (!data || data.event !== 'file_uploaded') {
18        return;
19      }
20
21      const job = {
22        id: Date.now().toString(),
23        fileId: data.fileId,
24        originalFileName: data.originalFileName || null,
25        userId: data.userId,
26        userEmail: data.userEmail,
27        status: data.scheduledAt ? 'scheduled' : 'pending',
28        scheduledAt: data.scheduledAt || null,
29        createdAt: getLocalISO(),
30      };
31
32      try {
```

```

33     await container.items.upsert(job);
34   } catch(upErr) {
35     throw upErr;
36   }
37 } catch(err) {
38   try {
39     await receiver.abandonMessage(message);
40   } catch(e) {
41     // Obsługa błędu komunikacji z repozytorium kolejki
42   }
43 }
44 };
45
46 const errorHandler = (error) => {
47   console.error('EVENT ERROR:', error);
48 };
49
50 receiver.subscribe({
51   processMessage: messageHandler,
52   processError: errorHandler
53 });
54 }

```

Listing 8: Implementacja mechanizmu nasłuchiwania kolejki (printQueueListener.js)

5.4.3 Punkty końcowe HTTP API

W tabeli 5 zestawiono punkty końcowe udostępniane przez interfejs sieciowy usługi *Queue Service*.

5.5 Printer Service

5.5.1 Opis ogólny

Usługa *Printer Service* stanowi centralny komponent orkiestracji systemu, odpowiedzialny za zaawansowane harmonogramowanie, weryfikację stanów urządzeń oraz nadzór nad realizacją procesów wytwórczych. Mikroserwis ten został zaimplementowany w języku TypeScript i składa się z trzech autonomicznych podsystemów logicznych:

Tabela 5: Punkty końcowe usługi Queue Service.

Metoda	Ścieżka	Opis
GET	/queue	Pobranie spaginowanej listy zadań z możliwością definiowania filtrów wyszukiwania.
GET	/queue/next	Pobranie kolejnego zadania przeznaczonego bezpośrednio do realizacji (zwraca kod 204 No Content w przypadku braku pozycji).
PATCH	/queue/cancel/:id	Anulowanie wybranego zadania drukowania (dostęp ograniczony do roli administratora).
GET	/queues	Pobranie metryk i informacji o stanie kolejek w ramach usługi <i>Azure Service Bus</i> (dostęp dla administratora).
GET	/health	Punkt końcowy diagnostyki stanu działania mikroserwisu (ang. <i>health check</i>).

- **Harmonogram zadań** (uruchamiany cyklicznie z interwałem jednogodzinowym) — odpowiada za sekwencyjny wybór kolejnego zadania produkcyjnego przeznaczonego do realizacji na podstawie kryteriów czasowych.
- **Monitor gotowości urządzenia** (uruchamiany cyklicznie z interwałem pięciominutowym) — nadzoruje i weryfikuje statusy zadań oczekujących na manualne potwierdzenie gotowości drukarki, egzekwując zdefiniowane limity czasowe (ang. *timeouts*).
- **Odbiornik telemetrii** (ang. *Telemetry Listener*) — asynchroniczny komponent zintegrowany z usługą *Azure Event Hubs*, którego zadaniem jest ciągły odbiór zdarzeń telemetrycznych transmitowanych przez agenta sprzętowego oraz bieżąca aktualizacja stanów zadań w bazie danych.

5.5.2 Mechanizm harmonogramowania

W ramach każdej minutowej iteracji usługa harmonogramująca wykonuje sekwencję operacji kontrolno-decyzyjnych:

1. **Weryfikacja aktywnego procesu:** System sprawdza, czy w bazie danych istnieje

zadanie o statusie printing. W przypadku wykrycia aktywnego procesu drukowania, bieżąca iteracja harmonogramu zostaje natychmiast przerwana.

2. **Weryfikacja stanu oczekiwania:** System kontroluje, czy proces nie został wstrzymany przez zadanie oczekujące na potwierdzenie fizycznej gotowości drukarki. Jeżeli takie zadanie istnieje, wybór kolejnej pozycji z kolejki zostaje pominięty.
3. **Wybór i alokacja zasobu:** W przypadku braku blokad, system wyszukuje zadanie o statusie pending lub scheduled o najwcześniejszej sygnaturze czasowej (scheduledAt), a następnie dokonuje atomowej zmiany jego statusu na waiting_for_printer_ready.

Na listingu 9 zaprezentowano implementację logiki wyboru oraz rezerwacji zadania, z uwzględnieniem dialektu zapytań SQL specyficznego dla bazy danych *Azure Cosmos DB*.

```
1  const query = '  
2    SELECT TOP 1 * FROM c  
3    WHERE (c.status = 'scheduled' OR c.status = 'pending')  
4    AND (c.scheduledAt <= "${now}"  
5    OR IS_NULL(c.scheduledAt)  
6    OR NOT IS_DEFINED(c.scheduledAt))  
7    ORDER BY c.scheduledAt ASC  
8  ';  
9  const { resources: jobs } = await container.items.query(query).  
    fetchAll();  
10  
11  for (const job of jobs) {  
12    if(!job.id) continue;  
13  
14    job.status = 'waiting_for_printer_ready';  
15    job.waitingStartedAt = nowDate;  
16  
17    await container.item(job.id, job.id).replace(job);  
18  }
```

Listing 9: Implementacja mechanizmu wyboru i rezerwacji zadań z kolejki (startScheduledJobs.ts)

5.5.3 Monitor gotowości urządzenia

Monitor gotowości odpowiada za stałe nadzorowanie zadań znajdujących się w stanach `waiting_for_printer_ready` oraz `delayed`. Komponent ten realizuje wielopoziomowy, deterministyczny proces przypomnień i eskalacji stanów:

- **Po 60 minutach** — następuje wyzwolenie i wysłanie pierwszego powiadomienia przypominającego.
- **Po 3 godzinach** — następuje wysłanie drugiego powiadomienia przypominającego.
- **Po 12 godzinach** — system dokonuje automatycznej zmiany statusu zadania na `delayed`.
- **Po 48 godzinach** — proces oczekiwania zostaje przerwany, a zadanie otrzymuje status terminalny `cancelled`.

Dystrybucja komunikatów alarmowych i powiadomień realizowana jest za pośrednictwem uniwersalnego mechanizmu sieciowego typu `Webhook`, którego adres docelowy jest definiowany w konfiguracji systemu poprzez zmienną środowiskową `ALERT_WEBHOOK_URL`.

5.5.4 Odbiornik telemetry

Usługa *Printer Service* konsumuje strumień danych telemetrycznych za pośrednictwem punktu końcowego zgodnego ze specyfikacją *Azure Event Hubs*, który jest natywnie udostępniany przez usługę *Azure IoT Hub*. W tabeli 6 zestawiono typy zdarzeń telemetrycznych generowanych przez agenta sprzętowego oraz powiązane z nimi akcje modyfikacji dokumentów w bazie danych *Azure Cosmos DB*.

Tabela 6: Zdarzenia telemetryczne przetwarzane przez usługę Printer Service.

Zdarzenie	Akcja w bazie danych Azure Cosmos DB
print_started	Zmiana statusu zadania na printing oraz rejestracja sygnatury czasowej w polu startedAt.
print_progress	Sukcesywna aktualizacja metryk postępu w polach progress, printTime oraz printTimeLeft.
print_completed	Zmiana statusu na completed wraz z zapisem czasu zakończenia (completedAt) i całkowitego czasu trwania procesu (printDuration).
print_failed	Zmiana statusu na failed oraz utrwalenie kodu błędu lub przyczyny awarii w polu failureReason.
print_cancelled	Zmiana statusu na cancelled oraz rejestracja momentu przerwania pracy w polu cancelledAt.

5.5.5 Punkty końcowe HTTP API

W tabeli 7 zestawiono i opisano punkty końcowe (ang. *endpoints*) wystawione przez interfejs sieciowy usługi *Printer Service*.

Tabela 7: Punkty końcowe usługi Printer Service.

Metoda	Ścieżka	Opis
GET	/printer/jobs	Pobranie spaginowanej listy zadań z obsługą dynamicznego filtrowania wyników.
GET	/printer/jobs/:id	Pobranie szczegółowych parametrów i metryk wybranego zadania.
GET	/printer/jobs/:id/progress	Pobranie bieżących informacji o postępie i szacowanym czasie zakończenia aktywnego procesu druku.

Tabela 7 – ciąg dalszy z poprzedniej strony

Metoda	Ścieżka	Opis
POST	/printer/jobs/:id/cancel	Wymuszenie przerwania i anulowania zadania drukowania (dostęp zastrzeżony dla administratora).
POST	/printer/jobs/:id/retry	Ponowne dodanie wybranego (np. przerwane) zadania do kolejki produkcyjnej.
POST	/printer/jobs/:id/confirm-ready	Zdalne potwierdzenie fizycznej gotowości urządzenia i stołu roboczego przez użytkownika.
DELETE	/printer/jobs/:id	Trwałe usunięcie rekordu zadania z bazy danych (dostęp zastrzeżony dla administratora).
GET	/printer/status	Odpytanie usługi <i>Azure IoT Hub</i> i pobranie aktualnego stanu operacyjnego fizycznej drukarki 3D.
GET	/printer/metrics	Pobranie zagregowanych metryk statystycznych systemowych zadań z podziałem na statusy.
GET	/health	Punkt końcowy diagnostyki stanu działania mikroserwisu (ang. <i>health check</i>).

5.6 Video Service

5.6.1 Opis ogólny

Usługa *Video Service* to mikroserwis zaimplementowany w środowisku uruchomieniowym Node.js z wykorzystaniem frameworka Express.js oraz języka JavaScript. Komponent ten odpowiada za tymczasowe przechowywanie aktualnego adresu URL strumienia wideo z kamery systemu *OctoPrint* w pamięci operacyjnej serwera (ang. *in-memory storage*), co eliminuje konieczność ciągłego odpytywania bazy danych o wolnozmiennie parametry sieciowe.

Agent uruchamiany na mikrokomputerze Raspberry Pi rejestruje bieżący adres kamery (lokalny adres IP lub adres publiczny udostępniany za pośrednictwem tunelu sieciowego *Cloudflare*) po każdym zainicjowaniu systemu operacyjnego. Aplikacja kliencka (frontendowa) cyklicznie pobiera ten adres i renderuje dynamiczny strumień wideo bezpośrednio w interfejsie użytkownika w elemencie `<img>` z wykorzystaniem formatu MJPEG (*Motion JPEG*).

5.6.2 Punkty końcowe API

W tabeli 8 zestawiono i opisano punkty końcowe (ang. *endpoints*) udostępniane przez interfejs sieciowy usługi *Video Service*.

Tabela 8: Punkty końcowe usługi Video Service.

Metoda	Ścieżka	Opis
POST	/camera-url	Rejestracja lub aktualizacja nowego adresu URL strumienia wideo z kamery.
GET	/camera-url	Pobranie aktualnego adresu URL kamery w celu poprawnego wyrenderowania podglądu.
GET	/health	Punkt końcowy diagnostyki stanu działania mikroserwisu (ang. <i>health check</i>).

6 Implementacja frontendu

6.1 Struktura aplikacji

Aplikacja frontendowa zbudowana jest w React 18 z TypeScript, z użyciem Vite jako narzędzia deweloperskiego i bundlera. Routing realizuje React Router v6. Zarządzanie stanem globalnym (dane użytkownika, status drukarki, metryki) odbywa się przez Zustand z middleware `persist` (przechowywanie preferencji motywu i języka w `localStorage`).

Struktura katalogów:

- `src/components/` — komponenty wielokrotnego użytku: `Card`, `StatusBadge`, `ProgressBar`, `ConfirmDialog`, `Layout`, `LoadingSpinner`.
- `src/pages/` — strony: `HomePage`, `LoginPage`, `RegisterPage`, `DashboardPage`, `UploadPage`, `ProfilePage`, `PrintControlPage`, `AdminDashboard`.
- `src/services/api.ts` — klient Axios dla wszystkich serwisów.
- `src/store/useStore.ts` — globalny store Zustand.
- `src/i18n/` — internacjonalizacja (`i18next`), tłumaczenia PL/EN.
- `src/types/` — typy TypeScript (`User`, `Job`, `PrinterStatus` itd.).
- `src/utils/formatters.ts` — funkcje pomocnicze (formatowanie dat, rozmiaru, czasu trwania).

6.2 Klient API

Komunikacja z backendem odbywa się przez klasę `ApiClient` w pliku `api.ts`, która tworzy osobne instancje Axios dla każdego serwisu (`Auth`, `User`, `Printer`, `Files`). Interceptory Axios automatycznie dołączają token JWT do nagłówka `Authorization` i obsługują odświeżanie tokenu w przypadku odpowiedzi 401:

```
1 client.interceptors.response.use(  
2   response => response,  
3   async (error) => {  
4     if (error.response?.status === 401) {  
5       const refreshToken = localStorage.getItem('refreshToken');
```

```

6      if (refreshToken) {
7          const { data } = await authClient.patch(
8              '/auth/refresh',
9              { refreshToken }
10         );
11         localStorage.setItem('accessToken', data.accessToken);
12         localStorage.setItem('refreshToken', data.refreshToken);
13         error.config.headers.Authorization =
14             'Bearer ${data.accessToken}';
15         return axios.request(error.config);
16     }
17 }
18 return Promise.reject(error);
19 }
20 );

```

Listing 10: Interceptor automatycznego odświeżania tokenu — api.ts

6.3 Routing i ochrona tras

6.4 Ekran aplikacji

6.4.1 Strona główna

6.4.2 Dashboard użytkownika

6.4.3 Upload i planowanie

6.4.4 Kontrola druku

6.4.5 Panel administracyjny

6.5 Internacjonalizacja

6.6 Obsługa motywu

7 Agent Raspberry Pi - AddiPi Agent

7.1 Opis ogólny

7.2 Architektura agenta

7.3 Połączenie z IoT Hub

7.4 Obsługiwane metody bezpośrednie

7.5 Przebieg obsługi metody startPrint

7.6 Telemetria

7.7 Komunikacja z OctoPrint

8 Obudowa urządzenia - projekt CAD

8.1 Cel i zakres projektu mechanicznego

8.2 Moduł CASE - obudowa Raspberry Pi z kamerą

8.2.1 Podstawa projektowa

8.2.2 Zakres modyfikacji

8.2.3 Mocowanie kamery

8.2.4 Struktura plików CASE

8.3 Moduł STAND - podstawka regulowana

8.4 Złożenie całości

8.5 Parametry wydruku

8.6 Instrukcja montażu

9 Konfiguracja Raspberry Pi

9.1 Opis środowiska

9.2 Instalacja agenta

9.3 Automatyczne uruchamianie agenta przez systemd

9.4 Tunel Cloudflare - ekspozycja kamery

9.4.1 Instalacja cloudflared

9.4.2 Skrypt publikowania URL kamery

9.4.3 Cykliczne odnawianie URL

9.5 Konfiguracja crontab

9.5.1 Uzasadnienie parametrów

9.6 Problem z dostępnością sieci przy starcie

9.7 Podsumowanie konfiguracji Raspberry Pi

10 Wdrożenie i infrastruktura

10.1 Infrastruktura chmurowa

10.2 Inicjalizacja infrastruktury

10.3 Konteneryzacja - Dockerfiles

10.4 Docker Compose

10.5 Zmienne środowiskowe

10.6 Wdrożenie frontendu

10.7 Wdrożenie agenta

11 Testowanie systemu

11.1 Metodologia testowania

11.2 Testy jednostkowe - Auth Service

11.3 Testy integracyjne przez Postman

11.4 Test end-to-end - pełny przepływ druku

11.5 Testy wydajnościowe i obserwacje

12 Podsumowanie

12.1 Osiągnięcia

12.2 Napotkane trudności

12.3 Możliwości rozwoju

12.4 Wnioski

Literatura

- [1] Dassault Systèmes. SOLIDWORKS – 3D CAD Design Software. <https://www.solidworks.com>. Dostęp: 2026.
- [2] Docker Inc. Docker Documentation. <https://docs.docker.com>. Dostęp: 2026.
- [3] Gina Häußge. Octoprint – The snappy web interface for your 3D printer. <https://octoprint.org>. Dostęp: 2026.
- [4] Meta Platforms, Inc. React – The library for web and native user interfaces. <https://react.dev>. Dostęp: 2026.
- [5] Microsoft Corporation. Azure Blob Storage documentation. <https://learn.microsoft.com/en-us/azure/storage/blobs/>. Dostęp: 2026.
- [6] Microsoft Corporation. Azure Cosmos DB documentation. <https://learn.microsoft.com/en-us/azure/cosmos-db/>. Dostęp: 2026.
- [7] Microsoft Corporation. Azure IoT Hub documentation. <https://learn.microsoft.com/en-us/azure/iot-hub/>. Dostęp: 2026.
- [8] Microsoft Corporation. Azure Service Bus documentation. <https://learn.microsoft.com/en-us/azure/service-bus-messaging/>. Dostęp: 2026.
- [9] Microsoft Corporation. Typescript – JavaScript With Syntax For Types. <https://www.typescriptlang.org>. Dostęp: 2026.
- [10] OpenJS Foundation. Express – Fast, unopinionated, minimalist web framework for Node.js. <https://expressjs.com>. Dostęp: 2026.
- [11] OpenJS Foundation. Node.js – JavaScript runtime built on Chrome’s V8 engine. <https://nodejs.org>. Dostęp: 2026.
- [12] Poimandres. Zustand – Bear necessities for state management in React. <https://github.com/pmndrs/zustand>. Dostęp: 2026.
- [13] Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>. Dostęp: 2026.

- [14] Tailwind Labs Inc. Tailwind CSS – Rapidly build modern websites without ever leaving your HTML. <https://tailwindcss.com>. Dostęp: 2026.
- [15] Evan You et al. Vite – Next Generation Frontend Tooling. <https://vitejs.dev>. Dostęp: 2026.

Załączniki

A Konfiguracja środowiska deweloperskiego

A.1 Wymagania

A.2 Pierwsze uruchomienie

B Struktura repozytoriów

C Endpointy API - podsumowanie